



Building a Real-Time Recommendation Web App with Go, React, and PostgreSQL

subtitle

Bachelor's Thesis
Degree Programme in Computer Applications
Spring 2025
Hao Zhang

DP Degree Programme in Computer Applications
Author Hao Zhang Year 2025
Subject Building a Real-Time Recommendation Web App with Go, React, and PostgreSQL
Supervisors Dr. Jawad Yasin

ABSTRACT

Keywords

Pages xx pages

Glossary

ACID	Atomicity, Consistency, Isolation, Durability (Database transaction properties)
API	Application Programming Interface
Axios	Promise-based HTTP client for JavaScript
Bcrypt	Password Hashing Function
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete (Basic database operations)
DOM	Document Object Model
Gin	Go Web Framework
Git	Distributed Version Control System
Go	(Golang) Programming Language
GORM	Go Object-Relational Mapper
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVCC	Multi-Version Concurrency Control
ORM	Object-Relational Mapper
PostgreSQL	Object-Relational Database System
React	JavaScript library for building user interfaces
REST	Representational State Transfer
RFC	Request for Comments (Internet standards document)
Socket.IO	Real-time communication library
SQL	Structured Query Language
TypeScript	Statically typed superset of JavaScript
UI	User Interface
URI	Uniform Resource Identifier
VCS	Version Control System
VS Code	Visual Studio Code (Source-code editor)
WebSocket	Computer communications protocol for full-duplex communication channels
XSRF	Cross-Site Request Forgery

Zustand State management library for React

Table of Contents

1 Introduction 1

2 Foundational Technologies 2

 2.1 Backend System Technologies 2

2.1.1	Go Programming Language.....	2
2.1.2	Gin Web Framework.....	3
2.1.3	PostgreSQL Database.....	3
2.1.4	RESTful API Architecture.....	3
2.1.5	Socket.IO for Real-time Communication	4
2.2	Frontend System Technologies.....	4
2.2.1	React Library	4
2.2.2	TypeScript Language.....	4
2.2.3	Zustand State Management.....	5
2.2.4	Tailwind CSS	5
2.2.5	Axios HTTP Client	5
2.3	Security Mechanisms	6
2.3.1	JWT (JSON Web Token) Authentication.....	6
2.3.2	Bcrypt Password Hashing.....	6
2.4	Development Environment and Tools	7
2.4.1	Git Version Control System.....	7
2.4.2	Visual Studio Code (VS Code) Editor	7
3	Methods.....	错误!未定义书签。
4	Practical Part.....	11
5	Results.....	11
6	Summary	11
	References	12

Figures

未找到图形项目表。

Tables

未找到图形项目表。

Program codes

未找到图形项目表。

1 Introduction

An increasing number of people are welcoming pets into their homes today. While pets bring significant happiness, connecting with other pet owners can sometimes present challenges. Finding someone for a dog walk, arranging a cat playdate, or simply sharing experiences with others who understand pet ownership can be difficult. Although large social media platforms exist, they often lack specific features for the pet owner community. Recognizing this gap and being interested in practical web development provided the motivation for this thesis project.

The primary objective of this thesis is to explore the creation of an improved online space for pet owners. To achieve this, a web application named "PetPals" was developed. This project serves not only as an academic exercise but also as an effort to address a real-world problem by building a functional tool. The practical development of this complete application also aims to enhance programming skills.

"PetPals" is designed as a real-time recommendation web application specifically for pet owners. The core concept is to help pet lovers connect with each other locally. The application allows users to create profiles for themselves and their pets. Using location and shared interests (such as pet type and preferred activities), it assists users in finding suitable companions for pet activities, arranging playdates, and discovering or organizing local pet-friendly events. Key features include user and pet profiles, location-based matching, tools for managing connections (like friend requests), event creation capabilities, and a real-time chat function for easy communication.

It is important to note that PetPals is not intended to be a general social media platform like Facebook. Instead, it aims to provide a focused and useful service for a specific group: people who care for their pets and wish to build a community based on this shared interest. The platform facilitates real-world interactions and mutual support among nearby pet owners.

The development of PetPals utilizes Go for the backend services, React for the user interface (frontend), and PostgreSQL for data storage. The application is designed to be accessible on both mobile and desktop web browsers.

This project also addresses the following key questions:

1. What specific features and requirements are most important for an online platform that helps pet owners connect in real-time?
2. How can the technical structure (architecture) of such a web application be designed to effectively and quickly match pet owners based on their location, pets, and interests?
3. How can technologies like Go, React, PostgreSQL, and WebSockets work together effectively to build a responsive and interactive platform for pet owners?

Through the development of the PetPals prototype and the effort to answer these questions, this thesis aims to demonstrate how modern web technologies can be used to improve the social connections of pet owners. The project focuses heavily on practical development (approximately 70%) while also documenting the design choices, technical solutions, and findings (approximately 30%).

2 Foundational Technologies

This chapter details the core technologies selected for the development of the web application discussed in this thesis. The selection process considered the functional requirements aiming for a robust, scalable, and maintainable system. The following sections provide an overview of the chosen technologies for the backend and frontend systems, outlining their general characteristics and capabilities.

2.1 Backend System Technologies

The backend architecture is responsible for application logic, data management, and communication services.

2.1.1 Go Programming Language

Go, often referred to as Golang, is an open-source programming language initially developed at Google by Robert Griesemer, Rob Pike, and Ken Thompson, and publicly announced in 2009. It was designed to improve programming productivity in an era of multicore processors, networked machines, and large codebases. Go is a statically typed, compiled language known for its simplicity, efficiency, and strong built-in support for concurrent programming through goroutines and channels. These characteristics make it

well-suited for developing high-performance network services and backend systems (Go Team Documentation, n.d.).

2.1.2 Gin Web Framework

Gin is a high-performance HTTP web framework written in Go. It features a Martini-like API but claims significantly better performance, partly due to its use of HTTP router. Gin is designed to be minimalistic, providing essential tools for routing, middleware handling, rendering (JSON, XML, HTML), and managing request/response cycles without imposing strict structural constraints. Its focus on performance and relatively small footprint make it a popular choice for building RESTful APIs and web services in the Go ecosystem (Gin Web Framework Documentation, n.d.).

2.1.3 PostgreSQL Database

PostgreSQL is a powerful, open-source object-relational database system with over 30 years of active development, originating from the POSTGRES project at the University of California, Berkeley. It has earned a strong reputation for reliability, feature robustness, and performance. PostgreSQL supports standard SQL queries and offers many advanced features, including complex queries, foreign keys, triggers, updatable views, transactional integrity (ACID compliance), and multi-version concurrency control (MVCC). Its extensibility allows users to define their own data types, index types, and functional languages (PostgreSQL Official Documentation, n.d.).

2.1.4 RESTful API Architecture

Representational State Transfer (REST) is an architectural style, first defined by Roy Fielding in his 2000 doctoral dissertation, that outlines constraints for designing networked applications. It relies on a stateless, client-server communication protocol, typically HTTP. Key principles of REST include resource identification through URIs, manipulation of resources through standard representations (like JSON or XML), and the use of standard HTTP methods (GET, POST, PUT, DELETE) to perform actions on these resources. RESTful APIs are widely adopted for web services due to their simplicity, scalability, and loose coupling between client and server (Fielding, 2000).

2.1.5 Socket.IO for Real-time Communication

Socket.IO is a popular JavaScript library (with implementations in other languages like Go) that enables real-time, bidirectional, and event-based communication between web clients and servers. It primarily utilizes the WebSocket protocol but provides fallbacks to other methods like HTTP long-polling if WebSockets are not supported. Socket.IO offers additional features beyond raw WebSockets, including reliability mechanisms (like automatic reconnection), message buffering, broadcasting to specific clients or groups (rooms), multiplexing via namespaces, and simpler APIs for handling events (Socket.IO Documentation, 2025). These features facilitate the implementation of complex real-time functionalities in web applications.

2.2 Frontend System Technologies

The frontend constitutes the user-facing part of the application, rendered and executed within the user's web browser.

2.2.1 React Library

An early prototype of React, created by a Facebook software engineer named Jordan Walke and dubbed FaxJS, was originally introduced in 2011. In 2012, Walke completed the prototype and invented React, which Facebook and Instagram quickly adopted that same year. React became available for Ruby on Rails and Python applications after being open sourced in 2013. 2015 brought the release of React Native, an extension of React for mobile programming on Android and iOS. React is a JavaScript library for building user interfaces, particularly single-page applications. It employs a declarative, component-based approach where UIs are constructed from reusable pieces called components. React utilizes a concept known as the virtual DOM to optimize updates and rendering, generally leading to efficient performance (React Official Documentation, n.d.).

2.2.2 TypeScript Language

TypeScript is an open-source programming language developed and maintained by Microsoft. It was first released in 2012 as a response to the challenges of building large-scale applications with JavaScript. TypeScript is a strict syntactical superset of JavaScript, meaning any valid JavaScript code is also valid TypeScript code, but it adds optional static

typing, classes, and interfaces. The primary benefit of using TypeScript is the ability to catch errors during development through type checking, leading to more robust and maintainable code, especially in larger projects. It also enhances developer productivity through improved code completion, refactoring, and navigation in code editors (TypeScript Official Documentation, n.d.).

2.2.3 Zustand State Management

Zustand is a small, fast, and scalable state management library primarily designed for React applications. It emerged as a simpler alternative to more complex state management solutions like Redux. Zustand leverages React hooks and provides a minimal API for creating shared state stores that components can subscribe to. Its core philosophy emphasizes simplicity, minimal boilerplate, and flexibility, allowing developers to manage application state without extensive setup or imposing strict architectural patterns (*Introduction - Zustand*, n.d.).

2.2.4 Tailwind CSS

Tailwind CSS is a utility-first CSS framework created by Adam Wathan, first released in 2017. Unlike component-based frameworks (like Bootstrap or Material-UI) that provide pre-designed components, Tailwind provides low-level utility classes that let developers build completely custom designs directly within their HTML markup. This approach encourages building unique UIs without writing custom CSS and results in highly maintainable and scalable styling, as styles are co-located with the elements they apply to. It also helps keep final CSS bundle sizes small by purging unused styles during the build process (Tailwind CSS Documentation, 2021).

2.2.5 Axios HTTP Client

Axios is a popular, promise-based HTTP client library for JavaScript, usable in both browser and Node.js environments. It simplifies the process of making asynchronous HTTP requests (like GET, POST, PUT, DELETE) to interact with backend APIs. Key features include request and response interception (allowing modification or logging), automatic transformation of request and response data (e.g., to/from JSON), client-side protection against Cross-Site Request Forgery (XSRF), and straightforward cancellation of requests.

Its ease of use and robust feature set have made it a common choice for handling API communication in modern web applications (Axios Documentation, n.d.).

2.3 Security Mechanisms

Ensuring the security of user data and application integrity is paramount.

2.3.1 JWT (JSON Web Token) Authentication

JSON Web Token (JWT) is an open, industry standard (RFC 7519) method for securely representing claims between two parties. It defines a compact and self-contained way for transmitting information as a JSON object. This information can be verified and trusted because it is digitally signed. JWTs typically consist of three parts separated by dots (.), which are the Header, Payload, and Signature. The header usually identifies the algorithm used for the signature, the payload contains the claims (statements about an entity, like user identity, and additional data), and the signature is used to verify that the sender of the JWT is who it says it is and to ensure that the message wasn't changed along the way (JWT.io Introduction, n.d.). JWTs are commonly used for authentication and authorization in web applications, allowing for stateless session management.

2.3.2 Bcrypt Password Hashing

Bcrypt is a widely used password hashing function designed by Niels Provos and David Mazières, based on the Blowfish cipher. Its primary purpose is to securely store user passwords. Unlike simple hashing algorithms (like MD5 or SHA-1) which are fast and thus vulnerable to brute-force attacks, Bcrypt is intentionally slow. It incorporates a "salt" (random data added to the password before hashing) to prevent rainbow table attacks and includes a configurable "cost" factor (work factor) that increases the computational effort required to compute the hash, making brute-force attacks significantly more time-consuming and expensive. Securely hashing passwords with algorithms like Bcrypt is a critical security practice for any application handling user credentials (Provos & Mazières, 1999).

2.4 Development Environment and Tools

Efficient development relies on appropriate tools for code management, writing, and collaboration.

2.4.1 Git Version Control System

Git is a distributed version control system (VCS) created by Linus Torvalds in 2005. It is designed to handle everything from small to very large projects with speed and efficiency. Git allows developers to track changes in source code during software development, facilitating collaboration among multiple developers. Key features include branching and merging capabilities (allowing developers to work on different features or fixes in isolation), history tracking (making it possible to revert to previous versions), and distributed workflows (where each developer has a full copy of the repository). Git has become the de facto standard for version control in software development (*Git - Documentation*, n.d.).

2.4.2 Visual Studio Code (VS Code) Editor

Visual Studio Code, commonly known as VS Code, is a free source-code editor developed by Microsoft for Windows, Linux, and macOS. Launched in 2015, it quickly gained popularity among developers due to its performance, extensive feature set, and large ecosystem of extensions. Core features include support for debugging, embedded Git control, syntax highlighting for numerous languages, intelligent code completion (IntelliSense), code snippets, and code refactoring tools. Its extensibility allows users to customize the editor with themes, key bindings, and additional functionality through extensions covering virtually any programming language or workflow (*Documentation for Visual Studio Code*, n.d.).

3 Goals and Methods

This section covers both the goals that were defined for the thesis project, as well as the methods and methodologies that were used during the thesis writing and the PetPals application creation process. Additionally, the various defined criteria used to evaluate the extent to which the research questions were successfully answered are also explained.

3.1 Project Goals and Design Vision

As explained in the introduction to this thesis, the primary project goal focused on creating a functional web application prototype named "PetPals". This application aims to provide a dedicated online space for pet owners to connect based on location and shared interests, offering features such as user/pet profiles, matching/discovery, and real-time communication.

The created application uses many modern technologies and architectural patterns. In order to address the topic of Building a Real-Time Recommendation Web App with Go, React, and PostgreSQL, the following research questions were addressed:

1. What specific features and requirements are most important for an online platform that helps pet owners connect in real-time?
2. How can the technical structure (architecture) of such a web application be designed to effectively and quickly match pet owners based on their location, pets, and interests?
3. How can technologies like Go, React, PostgreSQL, and WebSockets work together effectively to build a responsive and interactive platform for pet owners?

3.2 Defining Success Criteria

Before the application development began, specific success criteria were defined for each research question to provide an objective basis for evaluating the project's outcomes.

3.2.1 Defining Success Criterion for Core Features and Requirements

Regarding the first research question concerning essential features, the success criterion was defined as follows:

The final PetPals application prototype must implement and demonstrate functional core capabilities. These include user profile creation, viewing, and basic editing, along with pet profile creation, viewing, and association with a user. Furthermore, a user discovery or matching mechanism must be present, allowing filtering based on location and distance (using a distance radius) and at least two distinct pet profile attributes, such as pet type or activity level. A real-time chat interface enabling text message exchange between

connected users is also required, as is basic connection management allowing users to view their established connections. The successful implementation and basic operational correctness of these fundamental features would signify that this research question has been adequately addressed for the scope of this prototype.

3.2.2 Defining Success Criterion for Matching Architecture Effectiveness

Regarding the second research question focused on architectural design for effective matching, the success criterion involved two aspects: functional correctness and architectural soundness. For functional correctness, the implemented matching functionality must correctly return a list of users based on user-specified criteria including location (within a defined radius) and selected pet profile attributes like type or personality. The relevance of these returned matches for predefined test scenarios will be confirmed via manual inspection during the evaluation phase. For architectural soundness, the application architecture, as detailed and implemented in the following chapter covering the application coding, must show a clear and logical separation of concerns. This separation should be evident between the frontend presentation layer (React), the backend API and business logic layer (Go/Gin), and the data persistence layer (PostgreSQL), thereby enabling modularity and maintainability.

3.2.3 Defining Success Criterion for Technology Stack Integration

For the third research question addressing the effective integration of the chosen technology stack (Go, React, PostgreSQL, WebSockets via Socket.IO), the success criterion was defined by showing successful end-to-end data flow for both standard and real-time operations. Regarding standard operations using RESTful principles, successful creation, retrieval, update, and deletion (CRUD) operations for core data entities like user and pet profiles must be shown. This involves verifying data flow from the React frontend (using Axios), through the Go backend (using Gin and GORM), to the PostgreSQL database, and back to the frontend display. For real-time operations using WebSockets, successful transmission and reception of messages in near real-time must be shown within the chat feature. This requires showing messages originating from one React client, passing through the Go backend (using the Socket.IO implementation), and being delivered to other connected React clients with very low latency observed in the user interface.

3.3 Ethics and Responsibility

In developing the PetPals prototype, ethical considerations, particularly concerning user data privacy, were recognized. The application handles potentially sensitive information including user profiles, pet details, user location (for matching), and private messages. Security mechanisms, such as JWT for authentication and Bcrypt for password hashing as introduced in the preceding chapter, were implemented to protect user accounts. It is recognized that a production-level application would require more complete privacy policies, clear user consent for location sharing and data usage, and potentially stronger data protection measures. This prototype, however, focuses on technical implementation and does not store or process real user data beyond what is necessary for demonstration purposes within the development environment. (Metana, n.d.)

3.4 Project Planning and Management

The development of the PetPals application followed an iterative development approach. While not strictly following a formal framework like Scrum due to the nature of an individual project, the process involved breaking down the required features into smaller, manageable tasks. Progress was monitored informally, allowing for flexibility and adaptation as development challenges arose. Version control was maintained throughout the project lifecycle using Git, with code hosted on a GitHub repository. This enabled tracking changes, managing different development branches (if necessary), and providing a backup of the codebase.

3.5 Tools & Additional Methodologies

To assist with the coding process, specific tools and additional methodologies were used. Visual Studio Code (VS Code) served as the primary Integrated Development Environment (IDE) for writing, debugging, and managing both the Go backend code and the React/TypeScript frontend code, using its large extension ecosystem. As mentioned in the previous section, Git and GitHub were utilised for version control, allowing change tracking and repository hosting. Standard code formatting practices were followed, such as using gofmt for Go code and potentially ESLint/Prettier configurations for the TypeScript/React frontend, aiming for code consistency. Furthermore, architectural patterns explained previously, including frontend-backend separation and the combined use of RESTful and WebSocket APIs, were intentionally applied during the design and implementation phases.

4 Practical Part

5 Results

6 Summary

References

- Axios Documentation. (n.d.). *Getting Started | Axios Docs*. Retrieved March 31, 2025, from <https://axios-http.com/docs/intro>
- Documentation for Visual Studio Code. (n.d.). Retrieved March 31, 2025, from <https://code.visualstudio.com/docs>
- Fielding, R. T. (2000). *Fielding Dissertation: CHAPTER 5: Representational State Transfer (REST)*. https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
- Gin Web Framework Documentation. (n.d.). *Documentation*. Gin Web Framework. Retrieved March 31, 2025, from <https://gin-gonic.com/docs/>
- Git—Documentation. (n.d.). Retrieved March 31, 2025, from <https://git-scm.com/doc>
- Go Team Documentation. (n.d.). *Documentation—The Go Programming Language*. Retrieved March 31, 2025, from <https://go.dev/doc/>
- Introduction—Zustand. (n.d.). Retrieved March 31, 2025, from <https://zustand.docs.pmnd.rs/getting-started/introduction>
- JWT.io Introduction. (n.d.). *JSON Web Token Introduction—Jwt.io*. JSON Web Tokens - Jwt.io. Retrieved March 31, 2025, from <https://jwt.io/introduction>
- Metana. (n.d.). *Secure Web Apps with Bcrypt & JWT: Password Hashing & Authentication*. Retrieved April 7, 2025, from <https://metana.io/blog/bcrypt-and-jwt-web-app-security/>
- PostgreSQL Official Documentation. (n.d.). *PostgreSQL: Documentation*. Retrieved March 31, 2025, from <https://www.postgresql.org/docs/>

Provos, N., & Mazières, D. (1999). *A Future-Adaptable Password Scheme*.

[https://harrymoreno.com/assets/greatPapersInCompSci/A_Future-Adaptable_Password_Scheme_-_provos_\(1999\).pdf](https://harrymoreno.com/assets/greatPapersInCompSci/A_Future-Adaptable_Password_Scheme_-_provos_(1999).pdf)

React Official Documentation. (n.d.). *React*. Retrieved March 31, 2025, from <https://react.dev/>

Socket.IO Documentation. (2025, March 3). *Introduction | Socket.IO*. <https://socket.io/docs/v4/>

Tailwind CSS Documentation. (2021, June 17). *Documentation—Tailwind CSS*.

<https://tailwindcss.com/docs>

TypeScript Official Documentation. (n.d.). *The starting point for learning TypeScript*. Retrieved March 31, 2025, from <https://www.typescriptlang.org/docs/>